BCD Single Digit Multiplier
(A Self-Directed Project)

Amaey Advait

February 2026

Digital Circuits Laboratory [EC29202](Group 29)

Department of Electronics and Electrical Communication Engineering

Aim

To design and realize a combinatorial logic circuit that performs Binary Coded Decimal (BCD) multiplication of two single digit numbers using only adders and fundamental logic gates.

Motivation

During the design of combinatorial circuits for addition and subtraction, a natural curiosity emerged in extending these principles to implement the other basic operations like multiplication and division in hardware. A key challenge quickly identified was the representation of the tens digit, since conventional BCD +6 correction logic is primarily suited only for outputs less than 20.

This limitation served as a motivation to explore other existing approaches to convert binary bit sequences to decimal numbers. By reviewing the relevant literature, it became evident that the problem was tractable within the given hardware constraints with a structured and focused design approach. This realization served as the primary motivation for undertaking the cicuit level implementation of a Binary Coded Decimal (BCD) multiplier.

Components Required

- Seven Segment Display **(LT541)** - 2 units
- Binary Coded Decimal Decoder **(IC7447)** - 2 units
- Quad 2-input AND gate **(IC7408)** - 6 units
- Quad 2-input OR gate **(IC7432)** - 2 units
- 4 bit binary Full Adder **(IC7483)** - 7 units
- 4 bit D-type register **(IC74173)** - 1 unit
- Octal D-type Flip Flop **(IC74374)** - 1 unit
- Hex Keypad

Theory

The designed circuit consists of four basic building blocks:-

1. **Memory:** The 74LS374 is an octal edge-triggered D-type flip-flop used for temporary data storage and transfer. It consists of eight D flip-flops with a common clock input, where data present at the inputs is latched on the rising edge of the clock. In the circuit designed, the hex keypad inputs are given to D_{1-4}. Then, Q_{1-4} is connected to D_{5-8} to create a sequential connection between the two 4 bit numbers stored and feed the outputs Q_{5-8} to the inputs of the IC74173.

   The 74LS173 is a 4-bit register which includes clocked D flip-flops with enable inputs that control when the data is loaded, as well as active-low output enable signals for controlling output availability. The inputs D_{1-4} are then stored in the output pins Q_{1-4} of the IC74173. Both IC's use the same clock/strobe input from the hex keypad display. A detailed timing diagram elucidating the flow can be found below:

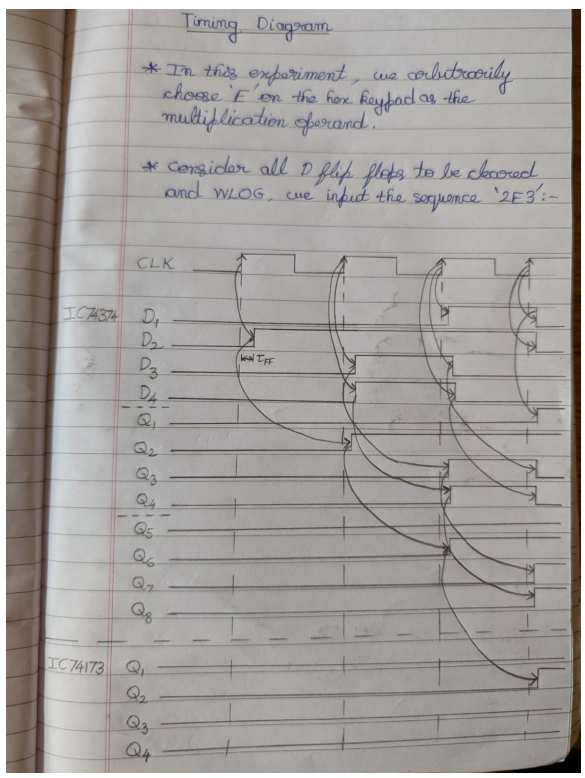

Figure 1: Timing Diagram of the memory block

2. **Binary Multiplication:** To compute the binary multiplication using combinatorial logic, four IC 7408's were used to compute the initial 16 partial products and these were then fed into the three IC7483's to obtain the 8 binary product bits $P_0 - P_7$.

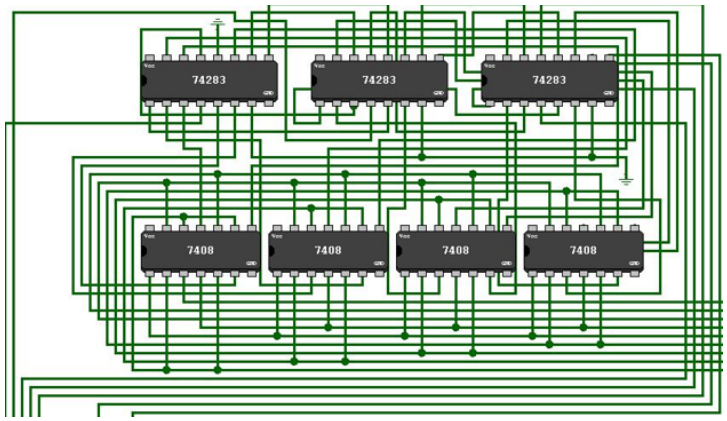

Figure 2: Binary Multiplication circuit.

Please note that the diagrams in this report uses IC74283 instead of IC 7483 as an equivalent substitute due to technical constraints in the inbuilt Logisim TTL IC options.

3. **Double Dabble Algorithm:**

The Double Dabble algorithm is a clever process used to convert binary integers into Binary Coded Decimal (BCD). The algorithm works by left-shifting the binary number bit-by-bit and adding 3 (+3) if the decimal value of the number in the units' place is greater than or equal to 5.

By doing so, it is essentially ensured that any value that would have reached 10 or more in the units' digit BCD representation correctly triggers a carry into the next higher nibble, because after every subsequent left shift (which effectively doubles the value), the added 3 becomes a 6, ensuring the base-16 overflow behaviour is retained.

In order to achieve this, four IC 7483's were used, each taking the set of inputs $P_{i+3}P_{i+2}P_{i+1}P_i \quad \forall i = 1, 2, 3, 4$.

On each set of product bits, the following logic to check if the decimal value of the number was greater than or equal to 5 was applied using two IC7408's and two IC7432's:-

$$\boxed{Y = P_{i+3} + P_{i+2}P_{i+1} + P_{i+2}P_i}$$

Tens	Units	Left Shift Column
-	-	0001 0101
-	0	001 0101
-	00	01 0101
-	000	1 0101
-	0001	0101
0	0010	101
00	0101	01
	+3	
00	1000	01
001	0000	1
0010	0001	
2	1	

Table 1: Implementation of the double dabble algorithm for $P_f = 21$

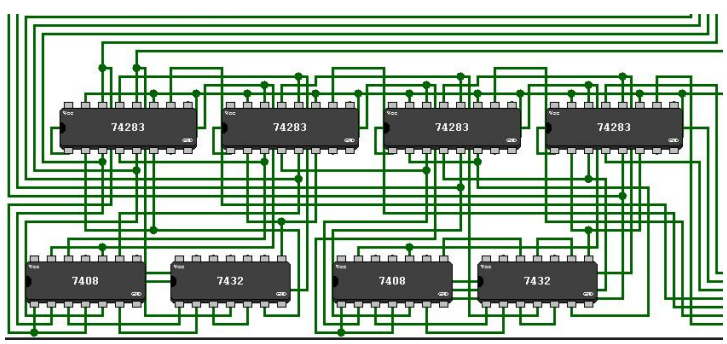

Figure 3: Double Dabble Algorithm Circuit Implementation

4. **Output:**

   The required BCD tens and units digit inputs are given to the respective IC7447's, which is then displayed on the seven segment displays.

The final circuit level diagram of the implementation is:-

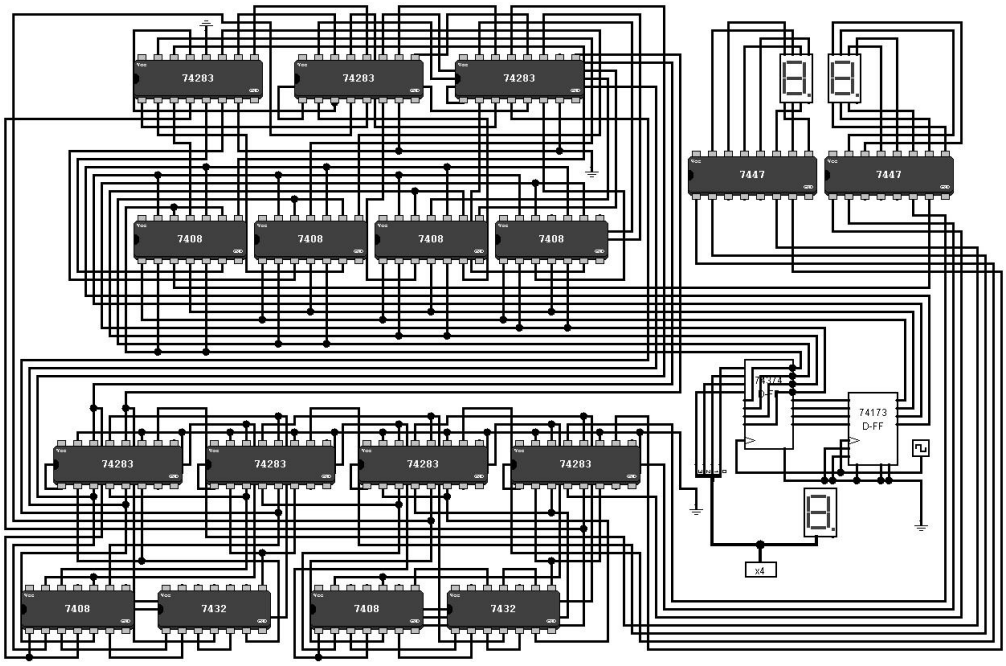

Figure 4: Circuit Level BCD single digit multiplier implementation

Observations

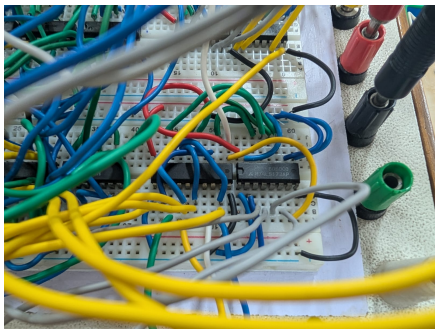

Figure 5: Memory Circuit

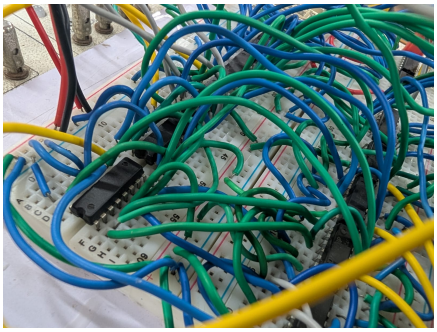

Figure 6: Binary Multiplication Circuit

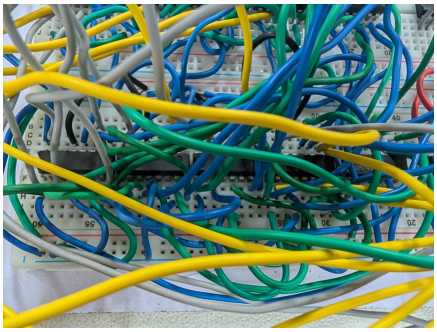

Figure 7: Double Dabble Algorithm circuit

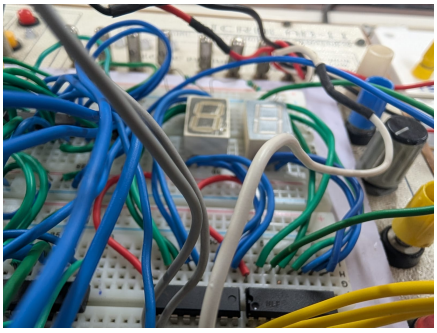

Figure 8: Output circuit

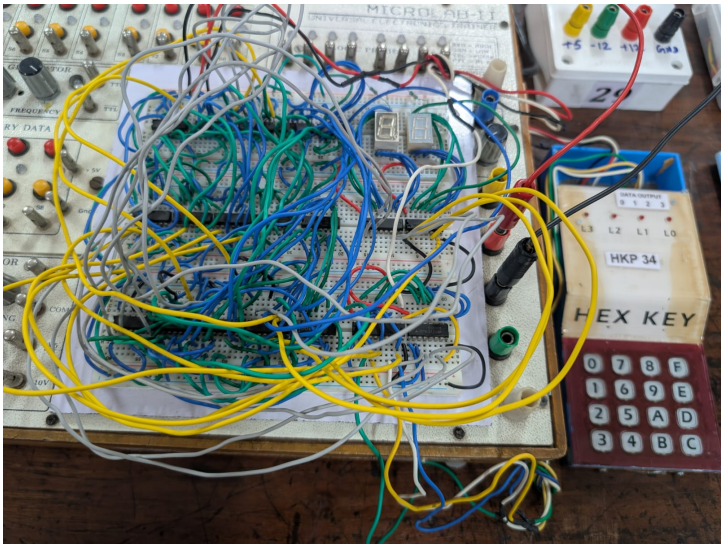

Figure 9: Complete hardware circuit

The circuit demonstrated consistent logical operations across a wide range of input combinations, with the output LEDs accurately displaying the product of the 4-bit operands. During the testing phase, an overload of the power rails lead to momentary glitches in the output, but this was negated by giving sufficient time between the experiments. The sequential memory portion of the circuit synchronized perfectly showing no sign of race conditions.

Results

The performance of the entire circuit was quantified through systematic testing at each block's output for each set of input operands. The observed output matched the theoretical product with 100% accuracy. This confirmed the integrity of the extensive combinatorial logic utilized in building the circuit.

Discussion

Through this experiment, a deep understanding of the logic that goes into building an arithmetic operand circuit from scratch was gained. The self-initiative project made me realize how energy consuming and inefficient the current methodology I employed is. Also, it was interesting for me to note that my breadboard was completely used up in just creating a single digit BCD multiplier circuit, with no space to pursue future directions of the same. Some simple future directions I was interested to pursue were:-

- A multiplication table that goes from 1x1, 1x2...upto 9x8, 9x9 built using a decade counter(IC7490) and an adder(IC7483) to input +1 at the required transition stages.
- Integrating addition, subtraction and multiplication (fundamental arithmetic operations) using 2 IC 74157's and their respective select logic.

In the current implementation, storing E in the IC74374 binary outputs Q_{5-8} feels redundant, but it is crucial to pursue the future directions. Regarding the double dabble logic, this formulation works empirically with just four adders because we constrain the inputs to be ≤ 9 and consequently, the output to be ≤ 81.

I hope to carry forward the knowledge I have gained through the verve and vigour that motivated me throughout this experience. As I've observed, the perfect result never comes in the first attempt, and I did spend hours "walking" across the breadboard, tapping with a wire connected to a logic probe and performing point-to-point validation of the 0/1 values shown, only to finally find a broken connecting wire lying in plain sight. But that only made me more confident to debug the circuit quicker the next time around. There is a certain unspoken joy that envelopes every engineer/scientist who works on a challenging problem just outside his/her reach and I would like to take this opportunity to thank Prof. Goutam Saha, Prof. Sharba Bandyopadhyay and all the amazing lab staff for graciously putting up with my incessant questions and requests while enabling that very joy that intrinsically accompanied the completion of this self-directed project.